

2009



GRUPO ESTUDIANTIL
INVENTRONICA

Joel Oswaldo Campos
Pérez

[CURSO BÁSICO DE PIC TABLAS]

En esta sección explicaremos que son las tablas, para que se usen y como se hacen. Se describe un ejercicio completo en el que se utiliza una tabla con el fin de comprender su funcionamiento y utilidad de manera práctica.

Contenido:

1) El contador de programa	3
2) Ejemplo con tabla (contador 7 segmentos)	4
a. Display 7 segmentos	4
b. Tabla de códigos 7 segmentos	4
c. Elaboración de tabla	6
d. Esquema del programa principal	7
e. Traducción a ensamblador	8
3) Ubicación de las tablas	10
4) Directiva DT (Definir Tabla)	12

1.- El Contador de Programa:

Como sabemos un programa en ensamblador está constituido por una serie de instrucciones que se ejecutan secuencialmente. Cada instrucción ocupa una posición de la memoria de programa, esta posición esta “apuntada” por un registro especial llamado **Contador de Programa**, este registro tiene una longitud de 13 bits <12:0> y está conformado por dos registros: PCL y PCLATH. PCL contiene los 8 bits menos significativos y PCLATH los 5 bits más significativos del Contador de programa, PCL es el registro más utilizado.

Poniendo un ejemplo, cuando en nuestro programa colocamos el ORG 0 (vector de reset) nos estamos posicionando en la posición 0 de la memoria de programa, es decir, PCL = 0. Veamos:

```

      ORG    0
      GOTO  inicio          ← PCL=0
      ORG    5
Inicio
      BCF    STATUS,RP0     ← PCL=5
      BSF    STATUS,RP1     ← PCL=6
      MOVLW   .7            ← PCL=7
      MOVWF   ADCON1        ← PCL=8
      BCF    STATUS,RP0     ← PCL=9

```

Como podemos ver, el PCL “lleva la cuenta” del número de instrucciones que ponemos en nuestro programa, si nosotros modificamos el valor del PCL durante la ejecución de nuestro programa NO modificamos la dirección de la instrucción donde estemos, sino que nos posicionamos hacia la dirección apuntada por PCL. Por ejemplo, la instrucción GOTO significa SALTO INCONDICIONAL hacia donde se le indique, realmente lo que la instrucción GOTO hace es modificar el valor de PCL con el valor indicado delante de la instrucción GOTO:

```

      nop                ← PCL
      decfsz  CONTA_1,F   ← PCL + 1
      goto   $-2          ← (PCL + 2) - 2 = PCL

```

El renglón que dice “goto \$-2” lo que hace en realidad es restarle al valor que tenga PCL el número 2 ($PCL \leftarrow PCL - 2$) con lo cual el PCL apunta ahora 2 lugares atrás. Como se puede ver, PCL es quien nos ubica dentro del programa, si modificamos su valor entonces saltamos hacia el lugar apuntado por PCL.

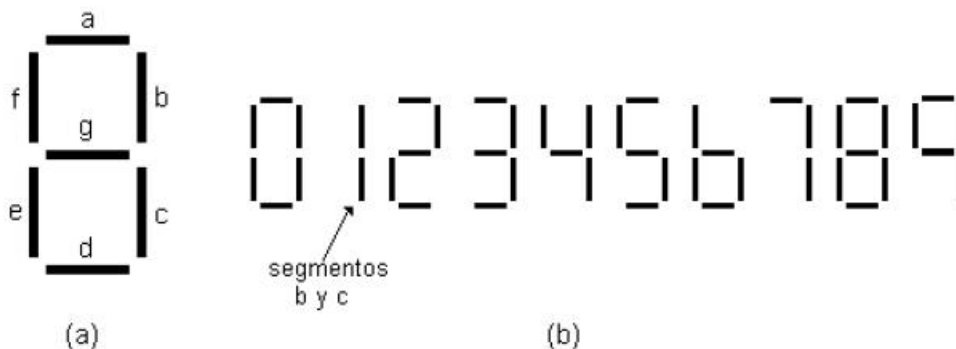
2.- Ejemplo con tabla (contador 7 segmentos)

Una tabla es como su nombre lo indica una subrutina que contiene una tabla de valores de retorno, es decir, le damos un valor a la tabla y nos regresa con otro valor, esa es una razón por la que se usan, entre otras cosas, para decodificadores. Las tablas se hacen con ayuda del registro PCL como veremos más adelante.

En este ejemplo haremos un contador binario con una tabla que traduzca un número binario a código 7 segmentos para visualizarse en un Display 7 segmentos.

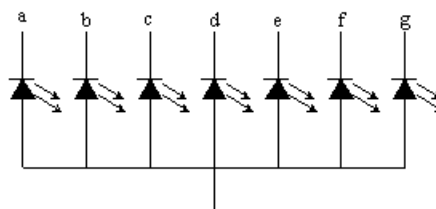
2.1 Display de 7 segmentos

El display 7 segmentos es un dispositivo que consta de un arreglo de leds dispuestos en un solo encapsulado de forma tal que podemos formar números del 0 al 9. Cada led es llamado "segmento" y se enlistan del "a" hasta el "g", algunos display contienen un octavo segmento correspondiente al punto. Vea la siguiente figura.



(a) Configuración de siete segmentos; (b) segmentos activos para cada dígito

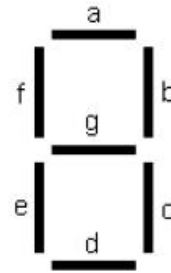
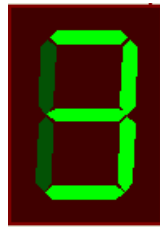
La siguiente figura muestra el diagrama de un display 7 segmentos de ánodo común, que es el que utilizaremos para este ejemplo:



2.2 Tabla de códigos 7 segmentos

Como se puede notar, para formar los números debemos encender los segmentos correspondientes a ese número, por ejemplo, si deseamos formar el número 3 debemos encender los siguientes segmentos:

a=0
b=0
c=0
d=0
e=1
f=1
g=0



Se debe recordar que el display utilizado es de ÁNODO COMÚN, lo que significa que los leds (segmentos) solo se encenderán conectándolos a tierra ó cero volts (0 lógico). De esta manera sabemos que el código 7 segmentos para formar el número 3 será: 0110000, donde el bit menos significativo corresponde al segmento "a". Así podemos generar una tabla con los valores de cada número en el display:

	= 11000000
	= 11111001
	= 10100100
	= 10110000
	= 10011001
	= 10010010
	= 10000010
	= 11111000
	= 10000000
	= 10010000

Hay que notar que el octavo bit corresponde al octavo segmento (punto), que no se visualiza aquí, y que no usaremos. Por tanto lo mantenemos apagado, es decir, en 1.

2.3 Elaboración de tabla

Ahora que conocemos los códigos en 8 bits de los números en 7 segmentos procedemos a realizar nuestro programa de **tabla**, que nos servirá como decodificador, observe como se utiliza el registro PCL:

tabla

addwf	PCL,F	;PCL $\leftarrow$ PCL + w
retlw	b'11000000'	; regresa con 0 en 7 seg
retlw	b'11111001'	; regresa con 1 en 7 seg
retlw	b'10100100'	; regresa con 2 en 7 seg
retlw	b'10110000'	; regresa con 3 en 7 seg
retlw	b'10011001'	; regresa con 4 en 7 seg
retlw	b'10010010'	; regresa con 5 en 7 seg
retlw	b'10000010'	; regresa con 6 en 7 seg
retlw	b'11111000'	; regresa con 7 en 7 seg
retlw	b'10000000'	; regresa con 8 en 7 seg
retlw	b'10010000'	; regresa con 9 en 7 seg

Observemos que tenemos una nueva instrucción: **retlw**, esta instrucción significa:

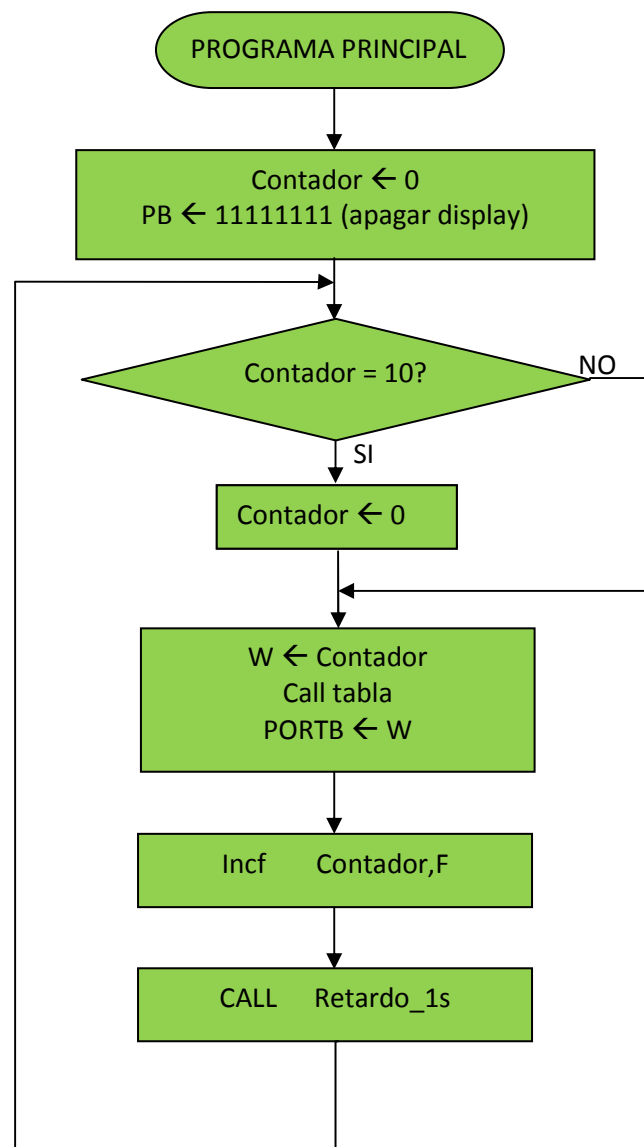
Retornar con $W = K$, donde K es la literal o valor denotado por b'xxxxxxx'

En la tabla, dependiendo del valor de W será el “tamaño del salto” dentro de la tabla a partir de la instrucción “**addwf PCL,f**”, y la instrucción **retlw** se encarga de retornar al programa principal asignándole primero a W el valor especificado en ese renglón. De esta manera si a W le asignamos el numero 0 y llamamos a la subrutina tabla entonces saltara 0 lugares y regresará con el valor b'11000000' en W; Si asignamos el 1 a W y llamamos la subrutina tabla entonces saltará 1 lugar a la instrucción “retlw b'11111001' y regresara con W=b'11111001'.

Es importante notar que debemos cuidar que el valor que le demos de entrada a W no sea mayor que el número de renglones de la tabla pues el programa no encontrará la instrucción retlw y no retornará y producirá un error.

2.4 Esquema del programa principal

Primero habremos de realizar el esquema de nuestro programa principal, debemos tomar en cuenta las condiciones iniciales, que el contador NO rebase el numero 9 y que el valor sea mostrado en el display conectado en el PORTB. La cuenta será cada segundo.



2.5 Traducción a ensamblador:

```

list    p=16f877a ; list directive to define processor
#include <p16f877a.inc> ; processor specific variable definitions

__CONFIG _CP_OFF & _WDT_OFF & _BODEN_OFF & _PWRTE_ON & _XT_OSC & _WRT_OFF & _LVP_OFF & _CPD_OFF

        CBLOCK    0x20
        CONTADOR
        ContA1
        ContA2
        ContA3
        ENDC
banco0  macro
        bcf        STATUS,RP0
        bcf        STATUS,RP1
        endm
banco1  macro
        bsf        STATUS,RP0
        bcf        STATUS,RP1
        endm

        ORG        0
        goto       inicio
        ORG        5
inicio  banco1
        movlw       d'7'
        movwf       ADCON1
        clrf        TRISB
        banco0

programa_principal
        movlw       b'11111111'
        movwf       PORTB
        clrf        CONTADOR

contar  movf         CONTADOR,W
        sublw       d'10'
        btfsc       STATUS,Z
        clrf        CONTADOR

        movf         CONTADOR,W
        call        tabla
        movwf       PORTB

        incf        CONTADOR,F
        call        Retardo_1s
        goto        contar

tabla  addwf         PCL,F
        retlw       b'11000000'
        retlw       b'11111001'
        retlw       b'10100100'
        retlw       b'10110000'
        retlw       b'10011001'
        retlw       b'10010010'
        retlw       b'10000010'
        retlw       b'11111000'
        retlw       b'10000000'
        retlw       b'10010000'

INCLUDE <RETARDOS_4MHZ.INC>
END

```

; definimos registros desde 0x20 en adelante
 ; contador lleva la cuenta
 ; conta1 2 y 3 son los registros para la rutina de retardo
 ; hacemos la macro para banco0
 ; hacemos la macro para banco1
 ; vector de reset
 ; ir al inicio del programa
 ; nos saltamos el ORG 4
 ; inicio del programa
 ; banco de configuraciones
 ; todos los pines digitales (ADCON1 <-- 7)
 ; portb ← salidas
 ; PB ← b'11111111' (apagamos display)
 ; CONTADOR ← 0
 ; Checamos que CONTADOR no pase del 9
 ; W ← CONTADOR
 ; W ← W-10
 ; Resultado de la operación fue 0? **NOTA***
 ; Sí fue 0, entonces limpia CONTADOR
 ; W ← CONTADOR
 ; llamamos tabla para decodificar
 ; PORTB ← W; desplegamos en display el número
 ; incrementamos contador
 ; llamamos retardo
 ; Se repite el ciclo
 ;tabla de decodificación binario a 7 segmentos
 ;regresa con 0 en 7 seg
 ;regresa con 1 en 7 seg
 ;regresa con 2 en 7 seg
 ;regresa con 3 en 7 seg
 ;regresa con 4 en 7 seg
 ;regresa con 5 en 7 seg
 ;regresa con 6 en 7 seg
 ;regresa con 7 en 7 seg
 ;regresa con 8 en 7 seg
 ;regresa con 9 en 7 seg

;librería de retardos
 END

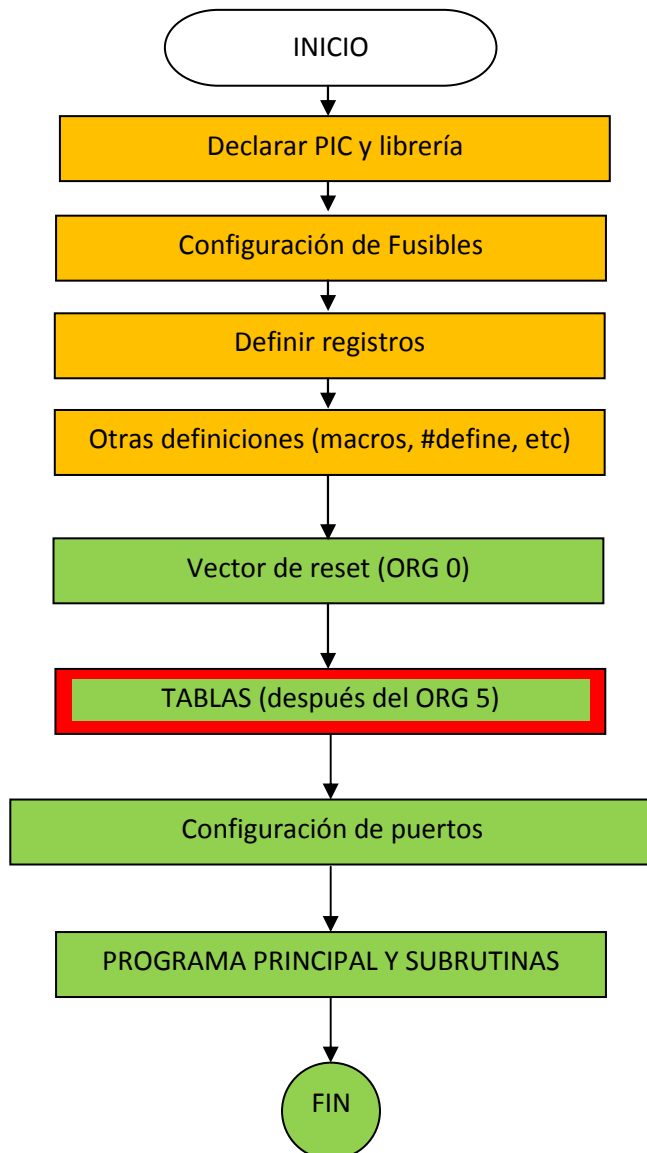
NOTA*: El bit STATUS,Z es una *bandera*, es decir, se pone a **1** automáticamente cuando el resultado de una operación fue CERO, en caso contrario la bandera STATUS,Z se mantiene apagada, es decir, en 0. Para hacer una comparación del tipo **“Contador = 10?”** en ensamblador se debe realizar haciendo una operación de resta:

- 1) Contador moverlo a W
- 2) A W restarle el numero 10
- 3) Preguntar si STATUS,Z = 1

En caso de que STATUS,Z=1 significa que el resultado de la resta dio 0 y por lo tanto el Contador es igual a 10, y así actuar en consecuencia, en este caso: si es 10 entonces regresarlo a 0, si no es 10 seguir con la cuenta normal.

3.- Ubicación de las tablas

Dado que estamos utilizando el PCL para las tablas debemos considerar que éste tiene 8 bits de longitud, lo que indica que el número máximo es 255, como es de esperarse debemos evitar que el PCL se desborde dentro de una tabla, es decir, las tablas no pueden superar la dirección 255 de la memoria de programa, lo cual puede ocurrir si el programa es largo y la tabla esta al final del programa. Así que el lugar más seguro para ubicar nuestras tablas será al inicio del programa, aquí no “estorban” y nos permite realizar el programa principal tan extenso como se requiera.



Recordemos, éste es el esquema general de un programa. Los recuadros naranjas son pre-configuraciones del micro. Los recuadros verdes son el programa en sí. Nótese como las tablas se ubican al principio del programa, antes de la configuración de puertos.

Ejemplo en ensamblador:

```

list    p=16f877a ; list directive to define processor
#include <p16f877a.inc> ; processor specific variable definitions

__CONFIG_CP_OFF & _WDT_OFF & _BODEN_OFF & _PWRTE_ON & _XT_OSC & _WRT_OFF & _LVP_OFF & _CPD_OFF
    CBLOCK    0x20
        CONTADOR
        ContA1
        ContA2
        ContA3
    ENDC
banco0 macro
    bcf        STATUS,RP0
    bcf        STATUS,RP1
endm
banco1 macro
    bsf        STATUS,RP0
    bcf        STATUS,RP1
endm

    ORG    0
    goto    inicio
    ORG    5

; tabla de decodificación binario a 7 segmentos
tabla
    addwf    PCL,F
    retlw    b'11000000' ; regresa con 0 en 7 seg
    retlw    b'11111001' ; regresa con 1 en 7 seg
    retlw    b'10100100' ; regresa con 2 en 7 seg
    retlw    b'10110000' ; regresa con 3 en 7 seg
    retlw    b'10011001' ; regresa con 4 en 7 seg
    retlw    b'10010010' ; regresa con 5 en 7 seg
    retlw    b'10000010' ; regresa con 6 en 7 seg
    retlw    b'11111000' ; regresa con 7 en 7 seg
    retlw    b'10000000' ; regresa con 8 en 7 seg
    retlw    b'10010000' ; regresa con 9 en 7 seg

inicio
    banco1
    movlw    d'7'
    movwf    ADCON1
    clrf     TRISB
    banco0

inicio_contador
    movlw    b'11111111'
    movwf    PORTB
    clrf     CONTADOR

programa_principal
    movf     CONTADOR,W
    sublw    d'10'
    btfsc    STATUS,Z
    clrf     CONTADOR
    movf     CONTADOR,W
    call     tabla
    movwf    PORTB
    call     Retardo_1s
    incf     CONTADOR,F
    goto     programa_principal

#include <RETARDOS_4MHZ.INC>
END

```

4.- Directiva DT (Define Table)

La directiva DT (Definir Tabla) nos sirve para hacer tablas de manera más corta y cómoda, nos permite escribir la tabla de manera horizontal, separando cada número con comas y evitándonos escribir reiteradamente la instrucción retlw.

EJEMPLO de tabla tradicional:

tabla

```
addwf PCL,F
retlw b'11000000' ; regresa con 0 en 7 seg
retlw b'11111001' ; regresa con 1 en 7 seg
retlw b'10100100' ; regresa con 2 en 7 seg
retlw b'10110000' ; regresa con 3 en 7 seg
retlw b'10011001' ; regresa con 4 en 7 seg
retlw b'10010010' ; regresa con 5 en 7 seg
retlw b'10000010' ; regresa con 6 en 7 seg
retlw b'11111000' ; regresa con 7 en 7 seg
retlw b'10000000' ; regresa con 8 en 7 seg
retlw b'10010000' ; regresa con 9 en 7 seg
```

EJEMPLO de tabla usando directiva DT:

```
tabla addwf      PCL,F
      DT      0xC0,0xF9,0xA4,0xB0,0x99,0x92,0x82,0xF8,0x80,0x90
```

Note que por conveniencia los números de la tabla están dados en su forma hexadecimal, por ejemplo, el numero binario b'11000000' es equivalente al C0 en hexadecimal (0xC0), la conversión binario-hexadecimal es una de las más simples de hacer.

Ejemplos de conversión BINARIO a HEXADECIMAL:

b'11000000' → 0xC0

b'11111001' → 0xF9

b'10100100' → 0xA4

GRUPO ESTUDIANTIL
INVENTRONICA



Aclaración de dudas respecto al tema
tratado en este trabajo al correo:

robo-sapiens@hotmail.com